Assignment 6 – CUDA Programming Report

Name: Divyam Puri

Roll Number: 102483023

Course: UCS645 – Parallel Computing

Question A – Device Query

Aim

To analyze GPU properties using CUDA device query.

Problem Description

The objective is to extract detailed information about the GPU such as:

- Compute capability
- Memory sizes
- Warp size
- Maximum threads and dimensions

This helps in understanding how to design efficient CUDA programs.

Terminal Snapshot

```
Device 0: Tesla T4
Compute Capability: 7.5
Max Threads per Block: 1024
Max Block Dimensions: 1024 x 1024 x 64
Max Grid Dimensions: 2147483647 x 65535 x 65535
Global Memory: 14912 MB
Shared Memory: 48 KB
Constant Memory: 64 KB
Warp Size: 32
Double Precision: Yes
```

Observations

GPU Name	Tesla T4
Compute Capability	7.5
Max Threads/Block	1024
Block Dimensions	1024x1024x64
Grid Dimensions	2147483647 x 65535 x 65535
Global Memory	14912 MB
Shared Memory	48 KB
Constant Memory	64 KB
Warp Size	32
Double Precision	Yes

Analysis

- Compute capability **7.5** indicates modern GPU architecture.
- Warp size = **32** → threads execute in groups of 32.
- Large grid size allows **massive parallel execution**.
- Shared memory is small but very fast → used for optimization.

Theory Answers

1. Architecture & Compute Capability

Tesla T4 with compute capability **7.5**.

2. Maximum Block Dimensions

1024×1024×64

3. Maximum Threads

Max Threads=GridDim×BlockDim

=65535×512=33,553,920

4. Why not use maximum threads?

- Resource contention
- Memory limits
- Thread divergence

- Overhead increases

5. Limiting Factors

- Shared memory
- Registers
- GPU memory
- Kernel complexity

6. Shared Memory

Fast on-chip memory → **48 KB**

7. Global Memory

Main GPU memory → **14912 MB**

8. Constant Memory

Read-only cache → **64 KB**

9. Warp Size

Group of threads executing together → **32**

10. Double Precision

Supported

Memory Usage

- Global memory stores large datasets
- Shared memory used for optimization
- Constant memory for fixed values

Conclusion

The Tesla T4 GPU provides strong parallel processing capabilities with efficient memory hierarchy and supports advanced CUDA features.

Question B – Array Sum

Aim

To compute the sum of array elements using GPU parallelism.

Problem Description

An array of floating-point numbers is generated and the sum is computed using CUDA threads.

Terminal Output

```
Sum = 1024
```

Methodology

- Allocate device memory
- Copy data to GPU
- Launch kernel
- Use `atomicAdd()` for accumulation
- Copy result back

Analysis

Each thread performs:

$$result = result + input[i]$$

Using `atomicAdd` ensures correctness but introduces slight overhead.

Observations

Input Size	Output
1024	1024

Memory Usage

- Global memory for array
- One variable for result

Conclusion

CUDA significantly speeds up reduction operations compared to sequential execution.

Question C – Matrix Addition

Aim

To perform parallel matrix addition using CUDA.

Problem Description

Two matrices A and B are added:

$$C[i][j]=A[i][j]+B[i][j]$$

Terminal Output

```
C[0] = 3
```

Methodology

- Flatten matrix into 1D array
- Assign one thread per element
- Perform addition in parallel

Calculations

Floating Operations:

$$n^2$$

Global Memory Reads:

$$2n^2$$

(A and B)

Global Memory Writes:

$$n^2$$

(C)

Analysis

- Highly parallelizable problem
- Performance limited by memory bandwidth
- Each thread works independently

Observations

Matrix Size	Output
512x512	Correct

Memory Usage

- Input matrices stored in global memory
- Output matrix also stored in global memory

Conclusion

Matrix addition achieves excellent parallel performance due to independent computations per thread.